Lab 1. Modelado OLTP → OLAP

Esquema estrella para PayTrack Colombia

Ceyni Paola Beltrán Hoyos y Carlos Guillermo Zúñiga Méndez

SI7006 Almacenamiento y Procesamiento de Grandes Datos - Maestría en Ingeniería de Datos

Profesor: Andrés Sacre Alzate - Universidad EAFIT

23 de octubre de 2026 · Repositorio: github.com/caronte281197/big-data-labs-2026-2 (rama lab1)

1. Diagnóstico del OLTP (Parte A)

Para el diagnóstico usamos 50.000 transacciones del generador del curso, con 500 clientes, 8 comercios y 90 días de operación. Corrimos cada consulta dos veces con EXPLAIN ANALYZE, en PostgreSQL 16 sobre Docker y en un MacBook Pro con chip M5 Pro, y reportamos la segunda corrida (Tabla 1): en la primera, PostgreSQL puede tener que leer los datos del disco, y en la segunda ya los encuentra en memoria, así que el tiempo no incluye esa lectura inicial. Las mediciones son del 22 de octubre de 2026; como las consultas filtran con CURRENT_DATE, esa fecha define qué transacciones entran en las ventanas de 30 y 7 días. Los planes completos están en el repositorio, en labs/01-modelado/entrega/planes_oltp_50k.txt.

Tabla 1. Planes de ejecución de Q1, Q2 y Q3 sobre el OLTP (50.000 transacciones, segunda corrida).

Consulta	**Operaciones que dominan el plan**	Filas en el JOIN o el Sort	**Execution Time (ms)**
Q1. Monto por categoría, últimos 30 días	Bitmap Heap Scan con el índice de fecha (visita los 569 bloques), Hash Join con merchants, HashAggregate	16.262	6,4
Q2. Tasa de fraude por segmento	Seq Scan de toda la tabla de transacciones, Hash Join con customers, HashAggregate	50.000	13,4
Q3. Transacciones por día de la semana y hora, últimos 7 días	Bitmap Heap Scan con el índice de fecha (visita 565 de 569 bloques), Sort por día y hora calculados con EXTRACT, GroupAggregate	3.696	3,5

En Q1, el índice de fecha ubicó las 16.289 transacciones de los últimos 30 días, pero la lectura recorrió los 569 bloques, porque esas filas están repartidas por toda la tabla. Luego vinieron el Hash Join con merchants (1,7 ms) y la agrupación por categoría (1,8 ms). En 3FN ese JOIN no se puede evitar: la categoría está solo en merchants y cada transacción guarda apenas el código del comercio. Con 500.000 transacciones diarias, cada consulta tendría que unir y agrupar 15 millones de filas. Q2 es la más lenta porque no filtra por fecha: lee las 50.000 filas, las une con customers y agrupa por segmento, y el Hash Join se lleva la mitad del tiempo (6,6 de 13,4 ms). El segmento solo existe en customers, y el OLTP guarda únicamente el actual; si un cliente pasa de mass a premium, sus transacciones anteriores quedan contadas como premium. El costo de Q2 crece con toda la historia: un año son 182,5 millones de filas, 3.650 veces las de hoy, y en esa proporción la consulta tardaría unos 50 segundos.

En Q3, el índice ubicó las 3.704 transacciones de la última semana, pero la lectura recorrió 565 de los 569 bloques, y el paso más costoso fue el Sort (1,7 ms). El OLTP no guarda el día de la semana ni la hora, así que el plan los calcula con EXTRACT en cada fila y ordena por ese resultado antes de agrupar. Con un año de operación, la semana tendría 3,5 millones de filas, y el ordenamiento, que hoy ocupa 269 kB, ya no cabría en los 4 MB que PostgreSQL le asigna por defecto y tendría que usar el disco. En un año, la tabla de transacciones pasaría de 4,5 MB a unos 16 GB sin contar índices, y estas consultas competirían por disco, memoria y procesador con la misma base que registra unos 6 pagos por segundo. Las cifras a un año son extrapolaciones lineales; la sección 3 las contrasta con una prueba de 500.000 transacciones.

2. Decisiones de diseño del esquema estrella (Parte B)

El proceso de negocio que modelamos es el procesamiento de pagos: cada vez que un cliente le paga a un comercio con un método de pago. El grano de la tabla de hechos, fact_transactions, es una fila por transacción de oltp.transactions, aprobada o rechazada, y cada fila conserva su transaction_id original, que

permite rastrearla hasta el OLTP y evita que una transacción quede repetida. Escogimos el nivel más detallado porque las tres preguntas agrupan por cosas distintas (categoría, segmento, día y hora) y desde el detalle se puede llegar a cualquier resumen, pero no al revés. Guardamos también las rechazadas porque parte de los fraudes termina rechazada, y sin ellas la tasa de fraude de Q2 cambiaría.

Alrededor de fact_transactions hay cinco dimensiones, una por cada cosa que se pregunta de un pago: quién pagó, dónde, con qué, qué día y a qué hora (Tabla 2). dim_date y dim_time venían dadas; las otras tres salen del OLTP, y dim_payment_method agrega method_type, que no existe en el OLTP y permite comparar tarjetas, billeteras digitales y transferencias. No pasan al data warehouse el nombre ni el correo de los clientes, porque identifican a una persona y ninguna pregunta de análisis los usa, y el ingreso mensual pasa como rango (menos de 3 millones, 3 a 6, 6 a 10 y más de 10) en vez del valor exacto. El data warehouse lo va a consultar el equipo de BI, y la Ley 1581 de 2012 pide tratar los datos personales solo con una finalidad informada al titular y por personas autorizadas.

Tabla 2. Dimensiones del esquema estrella.

Dimensión (llave)	Qué describe	Atributos
dim_customer (customer_key)	Quién paga	customer_id, segment, city, income_band, customer_since; valid_from, valid_to e is_current para la historia (SCD Tipo 2)
dim_merchant (merchant_key)	Dónde se paga	merchant_id, merchant_name, category, city
dim_payment_method (payment_method_key)	Con qué se paga	method_name, method_type (card, digital_wallet o bank_transfer)
dim_date (date_key)	Qué día	full_date, year, quarter, month, week_of_year, day_of_week, day_name, is_weekend, is_holiday_co
dim_time (time_key)	A qué hora	hour, minute, period_of_day

Las medidas de fact_transactions son cuatro. amount_cop es aditiva: se puede sumar por cualquier dimensión. is_fraud e is_approved son marcas de sí o no que, sumadas como 1 y 0, cuentan los fraudes y las transacciones aprobadas, así que también son aditivas. fraud_risk_score es no aditiva, porque sumar puntajes de riesgo no da nada interpretable; se promedia o se toma el máximo, y queda vacía en las transacciones sin evento de fraude. No hay medidas semiaditivas, que aparecerían en una tabla de saldos diarios (se suman entre clientes, pero no entre días). La tasa de fraude de Q2 se calcula al consultar, dividiendo dos sumas, porque una tasa tampoco se puede sumar.

Las tres dimensiones que salen del OLTP usan una surrogate key, un entero que genera el data warehouse (customer_key, merchant_key, payment_method_key), y guardan aparte la llave natural (customer_id, merchant_id, method_name). La razón principal es la historia: con SCD Tipo 2 un mismo cliente puede tener varias filas en dim_customer, una por versión, así que customer_id se repite y no puede ser la llave primaria. Además, la llave entera ocupa menos y se compara más rápido que los códigos de texto del OLTP, que en la Parte A había que emparejar fila por fila en cada JOIN, y deja al data warehouse independiente de cómo el sistema de origen asigne o cambie sus códigos.

dim_customer es SCD Tipo 2 porque el segmento y la ciudad de un cliente cambian con el tiempo. Si se sobrescribieran, como hace el OLTP, un cliente que pasa de mass a premium se llevaría a premium todas sus transacciones y fraudes anteriores, y la tasa de fraude por segmento de Q2 mezclaría periodos; con la ciudad pasaría lo mismo cuando alguien se muda. Con SCD Tipo 2, cada cambio cierra la fila vigente y abre una nueva (valid_from, valid_to e is_current), y cada transacción se une a la versión que estaba vigente en su fecha. En esta carga inicial cada cliente tiene una sola versión, que rige desde su fecha de registro y queda abierta con valid_to en 9999-12-31. Q2 se une por customer_key sin filtrar is_current, porque ese filtro dejaría por fuera las transacciones asociadas a versiones antiguas.

3. Comparación de rendimiento OLTP frente a OLAP (Parte D)

Medimos las tres consultas sobre el esquema estrella con el método de la Parte A, también el 22 de octubre: dos corridas con EXPLAIN ANALYZE, de las que tomamos la segunda (Tabla 3). Después repetimos las mediciones de los dos modelos con 500.000 transacciones. Con el mismo plan, el tiempo de una consulta llegó a variar un 30 % entre corridas, así que solo le atribuimos al modelo o a un índice las diferencias que

vienen con un cambio de plan o que se repiten en todas las mediciones. Los planes y el script de los experimentos están en labs/01-modelado/entrega.

Tabla 3. Planes con 50.000 transacciones y tiempos de la segunda corrida con 50.000 y 500.000. Con 500.000, Q1 y Q2 leen la tabla completa en paralelo en los dos modelos.

Consulta	Plan en el OLTP	Plan en la estrella	ms con 50.000 (OLTP / estrella)	ms con 500.000 (OLTP / estrella)
Q1	Bitmap Heap Scan con idx_txn_created_at (569 bloques, 16.262 filas) y Hash Join por texto con merchants	Seq Scan de fact_transactions (50.000 filas) y dos Hash Join por enteros, con dim_date y dim_merchant	6,4 / 8,7	27,2 / 33,6
Q2	Seq Scan (50.000 filas) y Hash Join por texto con customers	Seq Scan (50.000 filas) y Hash Join por enteros con dim_customer	13,4 / 10,6	37,9 / 35,2
Q3	Bitmap Heap Scan (565 de 569 bloques, 3.696 filas) y Sort de 3.696 filas por EXTRACT (269 kB)	Index Scan con ix_fact_date, una búsqueda por día (59 bloques), y Sort de solo 162 filas (33 kB)	3,5 / 1,4	27,0 / 10,3

Q3 es donde la estrella gana con claridad: bajó de 3,5 a 1,4 ms con 50.000 transacciones y de 27,0 a 10,3 ms con 500.000. La primera razón está en la lectura. La carga insertó los hechos ordenados por created_at, así que las transacciones de cada día quedaron juntas, y con ix_fact_date el plan busca día por día y lee 59 bloques; en el OLTP la última semana está repartida por toda la tabla y el plan visita 565 de 569. La segunda está en el ordenamiento. El día de la semana y la hora ya vienen calculados en dim_date y dim_time, así que el plan agrupa primero y ordena solo 162 filas (33 kB), mientras el OLTP calcula EXTRACT en cada fila y ordena 3.696 (269 kB) antes de agrupar. Con 500.000 transacciones, ese Sort del OLTP ocupó 3,3 de los 4 MB que PostgreSQL le asigna, como anticipamos en la Parte A; el de la estrella siguió en 168 filas.

En Q2 los dos planes tienen la misma forma: sin filtro de fecha, leen las 50.000 filas, hacen un Hash Join con los 500 clientes y agrupan por segmento. Cambia la llave del JOIN, un entero en la estrella y un código de texto en el OLTP, y esa parte del plan bajó de 6,6 a 4,9 ms. El total pasó de 13,4 a 10,6 ms, una mejora pequeña que se repitió en todas las mediciones; con 500.000, la diferencia (37,9 contra 35,2 ms) quedó dentro de la variación. En Q1 la estrella fue más lenta: 8,7 contra 6,4 ms con 50.000 y 33,6 contra 27,2 ms con 500.000. Como el filtro de fecha quedó en dim_date, el plan lee todas las filas de fact_transactions y descarta dos tercios en el Hash Join con dim_date, mientras el OLTP descarta lo viejo desde la lectura porque created_at está en la misma tabla. La llave entera abarató el JOIN con el comercio (de 1,7 a 1,3 ms), pero no compensa ese JOIN adicional.

En los experimentos con índices solo ix_fact_date ayudó: sin él, Q3 pasó de 1,4 a 5,3 ms, porque leyó los 516 bloques de la tabla de hechos en vez de 59. ix_fact_customer e ix_fact_merchant no aparecen en ningún plan y al quitarlos ninguno cambió; servirían para consultas sobre un solo cliente o comercio, que no están entre las tres. Al obligar a Q2 a usar ix_fact_customer, tardó 12,8 ms e hizo 45.590 lecturas de bloque sobre una tabla de 516, porque las transacciones de cada cliente están repartidas por toda la tabla. Esa es la primera razón por la que PostgreSQL prefiere un Seq Scan: si la consulta necesita casi todas las filas, leer en orden cuesta menos que ir y volver entre el índice y la tabla. La segunda es que PostgreSQL decide con una estimación que, por defecto, le cobra a cada bloque leído fuera de orden cuatro veces lo de uno leído en orden (random_page_cost), un supuesto de discos mecánicos. Por eso en Q1 descartó buscar día por día en ix_fact_date, aunque ese plan, forzado, tardó 5,0 ms contra 8,6. Con random_page_cost en 1,1, el valor que la documentación de PostgreSQL sugiere para SSD, lo escogió solo y Q1 bajó a 3,9 ms, menos que en el OLTP; Q2 siguió con el Seq Scan.

Con diez veces más transacciones, los tiempos crecieron entre 3 y 8 veces, no 10. En Q1 y Q2, PostgreSQL repartió la lectura de la tabla grande entre tres procesos en los dos modelos (Parallel Seq Scan), y esas consultas crecieron entre 3 y 4 veces; Q3 no se repartió y creció unas 7,5 veces en los dos. El patrón se mantuvo: la estrella es 2,6 veces más rápida en Q3, queda pareja en Q2 y sigue perdiendo en Q1. Las cifras a un año de la Parte A son entonces un orden de magnitud y no un pronóstico: el reparto entre procesos baja la pendiente, pero tiene un tope (dos procesos de ayuda por defecto), y a partir de ahí el tiempo vuelve a crecer al ritmo de las filas.